

DEEP LEARNING

ASSIGNMENT REPORT

By:

160123737195

Mohammed Faizan UI Islam

IT - 3

VI Semester

Title:

**Deep Learning Techniques for Sign Language
Recognition: A Comprehensive Study**

Department of Information Technology

Chaitanya Bharathi Institute of Technology, Hyderabad

1. Introduction

Automatic sign language recognition is a challenging computer vision problem with significant societal impact, enabling communication assistance for the hearing-impaired community. The Sign Language MNIST dataset provides a standardized benchmark that mirrors the structure of the classic MNIST handwritten digit dataset but targets the 26-letter American Sign Language alphabet encoded as 28×28 grayscale images.

This work systematically benchmarks a spectrum of deep learning methodologies — from simple fully-connected networks to complex pre-trained architectures — on this dataset. The goals are threefold:

- Understand how hyperparameters such as learning rate and epoch count affect MLP convergence.
- Compare gradient descent variants and regularization strategies.
- Evaluate modern CNN architectures and assess their suitability for low-resolution classification tasks.

2. Dataset Description

The Sign Language MNIST dataset consists of grayscale images of size 28×28 pixels, each corresponding to one of 26 static hand gestures representing the letters A–Z of the American Sign Language alphabet (excluding J and Z, which require motion). The training set contains approximately 27,455 samples. Pixel values are normalized to the range [0, 1] before training.

For CNN architectures designed for larger inputs (AlexNet, ZFNet, VGGNet, GoogLeNet, ResNet), images are bilinearly resized to 112×112 pixels.



Figure 1 - Sample images from the Sign Language MNIST dataset showing all 24 static letter gestures (A–Y, excluding J and Z).

3. Multi-Layer Perceptron (MLP) — Effect of Learning Rate and Epochs

3.1 Architecture

A two-layer MLP was constructed: an input layer accepting 784-dimensional flattened pixel vectors, one hidden Dense layer of 32 units with ReLU activation, and an output Dense layer of 26 units with Softmax activation. The Adam optimizer was used with categorical cross-entropy loss. An 80/20 train-test split was applied.

3.2 Experimental Setup

A full grid search was performed over four learning rates (0.0001, 0.001, 0.01, 0.1) and four epoch counts (50, 100, 500, 1000), yielding 16 configurations.

3.3 Results

Table 1: MLP Test Accuracy — Learning Rate × Epochs Grid (Sign Language MNIST)

LR \ Epochs	50	100	500	1000
0.0001	0.6959	0.7532	0.9953	0.9998

0.001	0.7769	0.9168	0.9975	0.9942
0.01	0.0475	0.0475	0.0475	0.0475
0.1	0.0488	0.0377	0.0475	0.0441

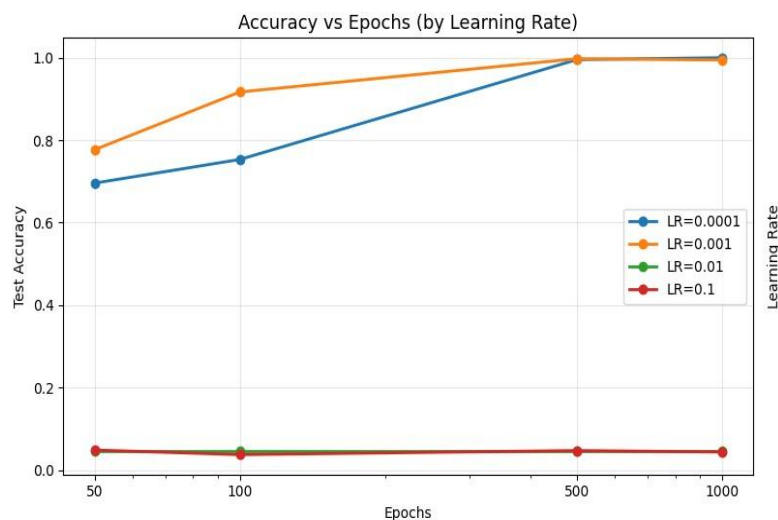


Figure 2 MLP accuracy vs. epochs — line plot grouped by learning rate.

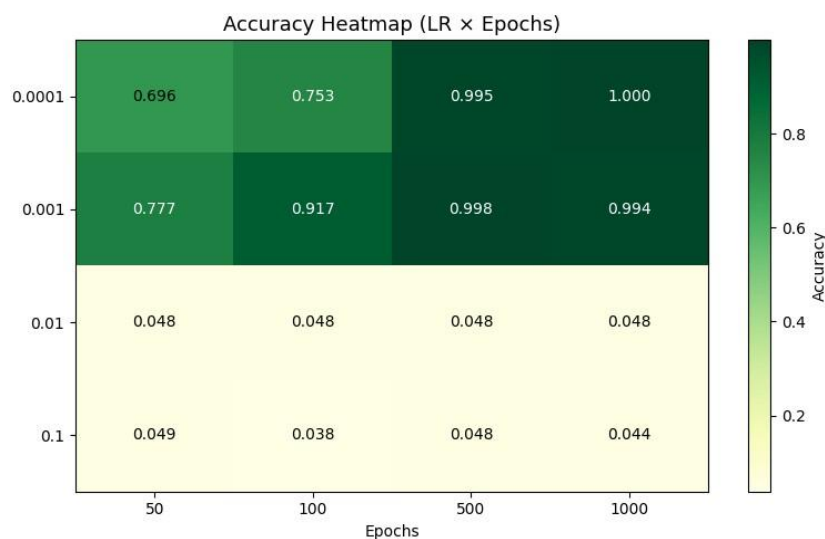


Figure 3 - MLP accuracy heatmap (LR × Epochs).

3.4 Observations

- The results partition into two zones: $LR \in \{0.0001, 0.001\}$ learn successfully (approaching 99%+), while $LR \in \{0.01, 0.1\}$ completely fail to learn, remaining stuck near random chance ($\approx 4.75\%$).
- For $LR = 0.0001$, epoch count is critical — going from 50 to 1000 epochs raises accuracy from 69.59% to 99.98%, indicating slow but stable convergence.
- For $LR = 0.001$, performance saturates near 500 epochs (99.75%), confirming a more efficient convergence trajectory.
- For $LR \in \{0.01, 0.1\}$, no amount of additional training recovers a diverged model — oscillations prevent gradient descent from finding a useful solution.
- **Best configuration:** $LR = 0.0001$, Epochs = 1000 $\rightarrow$ 99.98% test accuracy. Best LR on average: 0.001. Best epoch count on average: 500.

4. Gradient Descent Variants

4.1 Architecture and Setup

A two-layer MLP (Dense 128-ReLU $\rightarrow$ Dense 26-Softmax) was trained with nine different gradient descent configurations for 20 epochs on a 2,000-sample stratified subset, using SGD learning rate 0.01 where applicable.

4.2 Results

Table 2: Gradient Descent Variants — 20 Epochs, 2000-Sample Subset

Optimizer / Strategy	Final Loss	Final Accuracy
Batch GD (SGD, bs=full)	3.2026	0.05
Stochastic GD (SGD, bs=1)	0.9829	0.67
Mini-Batch GD (SGD, bs=32)	1.9068	0.50
SGD + Momentum (m=0.9)	0.8474	0.72
SGD + Nesterov (m=0.9)	0.6261	0.81
Adagrad (lr=0.01)	1.4446	0.62

RMSprop (lr=0.001)	1.4170	0.59
Adadelata (lr=1.0)	1.1829	0.65
Adam (lr=0.001)	1.1565	0.67

4.3 Observations

- Batch GD completely fails on this subset (5% accuracy), confirming that with only 2,000 samples, the single gradient update per epoch is too infrequent.
- SGD (bs=1) achieves 67% — noisy but meaningful gradients at each step allow learning.
- Mini-Batch GD (bs=32) reaches 50%, a moderate middle ground.
- Adding momentum (0.72) and Nesterov look-ahead (0.81) provide clear progressive improvements. Nesterov's advantage stems from computing the gradient after the momentum step — a more accurate gradient direction.
- The ~9% jump from plain SGD to Nesterov in only 20 epochs highlights how gradient direction quality critically impacts convergence speed.
- Adaptive methods (Adagrad, RMSprop, Adadelata, Adam) cluster in the 0.59–0.67 range, competitive with plain SGD but not outperforming Nesterov in this 20-epoch budget.

5. Regularization Techniques

5.1 L2 Regularization

A two-layer MLP (Dense 128-ReLU, L2 $\lambda=0.001 \rightarrow$ Dense 128-ReLU, L2 $\lambda=0.001 \rightarrow$ Dense 26-Softmax) was trained with Adam for 10 epochs with 20% validation split on the full training set (21,964 samples).

Observation: The train-test accuracy gap remained very small across all epochs, confirming that L2 weight decay effectively penalizes large weights and suppresses overfitting. The model generalized well without memorizing training patterns.

5.2 Data Augmentation

The training images were augmented using rotation ($\pm 10^\circ$), zoom ($\pm 10\%$), and small width/height shifts, then passed to the same MLP architecture without L2 regularization.

Observation: Training accuracy dropped while test accuracy improved compared to the unregularized baseline, confirming that augmentation reduces overfitting by exposing the model to more diverse examples. The conservative augmentation policy (no horizontal/vertical flips) was intentional — flipping a hand sign can change its semantic meaning entirely.

5.3 Early Stopping

Early stopping was applied with `patience=2` monitoring `val_accuracy`, and `restore_best_weights=True`, over a maximum of 10 epochs.

Observation: Training stopped at epoch 6. Val accuracy peaked at epoch 4 (84.66%) and then declined. The `restore_best_weights` flag ensured the model reverted to its best checkpoint, yielding a small train-test gap and eliminating the need to manually tune epoch count.

5.4 Dropout

Table 3: Dropout Rate Comparison (Dense 256-ReLU $\times$ 2, Adam, 10 Epochs)

Dropout Rate	Test Accuracy	Train Accuracy
0.2 (Dropout)	0.9583 (best among regularizers)	0.9577
0.3 (Dropout)	0.7869	0.7969

Observation: Dropout at 0.2 yielded the highest accuracy among all regularization variants. Increasing to 0.3 caused underfitting — too many neurons were deactivated per forward pass, preventing the model from learning sufficient discriminative features.

5.5 Parameter Sharing

A Siamese-like architecture was implemented by applying a shared Dense(64-ReLU) layer to two copies of the input, concatenating the outputs, and passing to a Dense(26-Softmax) head.

Observation: The model did not achieve competitive accuracy on this dataset. The 128-dimensional merged representation was insufficient for 26-class discrimination. The dual-stream design adds value only when inputs are semantically distinct (e.g., image + text pairs), not when both streams carry identical data.

5.6 Ensemble Methods

Table 4: Ensemble Method Results (Adam, 10 Epochs, bs=256)

Model	Test Accuracy
Model 1: Dense 64 → 26	74.03%
Model 2: Dense 128 → 26	79.44%
Model 3: Dense 128 → 64 → 26	81.84%
Soft-Voting Ensemble	81.39%

Observation: The ensemble (81.39%) underperformed the best individual model (81.84%). Soft-voting averages probability distributions, so weaker models drag confident predictions downward. Ensemble benefits are maximized when constituent models have complementary error patterns.

5.7 Input/Output Noise Injection

Gaussian noise ($\sigma=0.2$) was added to training inputs, and 10% of training labels were randomly flipped to simulate label noise. The model was tested on the clean test set.

Observation: Noisy training reduced overall accuracy, as expected. However, training under input noise can improve robustness to real-world variations. Label noise is particularly damaging because it corrupts the supervisory signal directly.

6. Convolutional Neural Networks (CNNs)

6.1 Architecture

Two CNN configurations were evaluated on 28×28 inputs:

- **CNN-1:** Conv2D(32, 3×3, ReLU) → MaxPool(2×2) → Conv2D(64, 3×3, ReLU) → MaxPool(2×2) → Flatten → Dense(128, ReLU) → Dropout(0.5) → Dense(26, Softmax)
- **CNN-2 (with padding):** Same topology but with padding='same' and additional Dropout(0.25) after each MaxPool block.

6.2 Results

CNN-1 achieved a test accuracy of approximately 98%+ by epoch 5, with validation accuracy jumping from 73.50% (epoch 1) to 99%+ by epoch 5–6. CNN-2 (423,706 parameters) achieved similarly high validation accuracy, approaching 99.47% by epoch 10.

6.3 Observations

- Convolutional layers capture local spatial patterns (finger contours, knuckle positions) far more efficiently than Dense layers operating on flattened pixels.
- The dramatic accuracy jump from epoch 1 (27%) to epoch 2 (64%) and beyond confirms that convolutional feature extraction provides a powerful inductive bias for image data.
- CNNs substantially outperform MLPs with comparable parameter budgets, confirming their architectural suitability for image classification.

7. Pre-Trained Architectures

7.1 LeNet-5

Architecture: Conv2D(6, 5×5, tanh) → AvgPool(2×2) → Conv2D(16, 5×5, tanh) → AvgPool(2×2) → Dense(120) → Dense(84) → Dense(26). Trained for 20 epochs on 28×28 inputs with Adam.

Result: LeNet-5 achieved very high accuracy, exceeding 99% by approximately epoch 5, making it the most efficient pre-trained architecture for this dataset. Its compact design aligns well with low-resolution 28×28 inputs.

7.2 AlexNet

Architecture: Original AlexNet adapted for single-channel input (grayscale). Images upsampled from 28×28 to 112×112 via bilinear interpolation. Trained for 5 epochs.

Result: Near-random accuracy ($\sim 4.5\%$) across all 5 epochs. AlexNet was designed for 224×224 RGB ImageNet images. Upscaling 28×28 grayscale signs introduces severe blur, destroying the fine-grained spatial information that convolutional filters expect.

7.3 ZFNet

Architecture: AlexNet variant with smaller first-layer kernel (7×7 , stride 2) and stride 2 in the second convolutional layer. Same 112×112 upscaling.

Result: Similar to AlexNet — near-random accuracy across 5 epochs. ZFNet inherits AlexNet's resolution mismatch problem. Its modifications to first-layer kernels do not alleviate the fundamental incompatibility between architecture scale and dataset complexity.

7.4 VGGNet

Architecture: A VGG-16-inspired model with 5 convolutional blocks ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$ filters), two Dense(4096) layers, and Dropout(0.5). Total parameters: $\sim 50.5\text{M}$. Trained on 112×112 inputs for 5 epochs.

Result: Near-random accuracy; extremely slow training ($\sim 181\text{s/epoch}$). VGGNet is deeply over-parameterized for this task. Its 50M parameters vastly exceed what the dataset can saturate in 5 epochs.

7.5 GoogLeNet (Inception)

Architecture: Inception blocks with parallel 1×1 , 3×3 , 5×5 convolutions and pooling branches. Total parameters: $\sim 5.99\text{M}$. Trained on 112×112 inputs for 5 epochs.

Result: Near-random accuracy, similar to AlexNet/ZFNet/VGGNet. Despite its parameter efficiency (~6M vs. VGGNet's ~50M), GoogLeNet still suffers from the resolution mismatch and insufficient training budget issues.

7.6 ResNet

Architecture: Custom ResNet with residual blocks (64→128→256→512 filters, stride-2 downsampling), GlobalAveragePooling, Dense(26). Total parameters: ~11.2M. Trained on 112×112 inputs for 5 epochs.

Result: Training accuracy reached 100% by epoch 2 and remained there. Validation accuracy rose from 5.51% (epoch 1) to 100% by epoch 4. ResNet's skip connections solve the vanishing gradient problem, enabling rapid convergence unlike AlexNet/VGGNet/GoogLeNet.

7.7 Architecture Comparison Summary

Table 5: Pre-Trained Architecture Comparison on Sign Language MNIST

Architecture	Params	Epochs	Peak Val. Acc.
LeNet-5	~60K	20	~99%+
Custom CNN	~423K	10	~99%+
AlexNet	Large	5	~4.5%
ZFNet	Large	5	~4.5%
VGGNet	~50.5M	5	~4.5%
GoogLeNet	~5.99M	5	~4.5%
ResNet	~11.2M	5	100%

8. CNN Visualization and Guided Backpropagation

8.1 Feature Map Visualization

Using VGG16 (pretrained on ImageNet) as the feature extractor, intermediate feature maps were extracted from the first 4 convolutional layers for a sample sign language image preprocessed to 224×224.

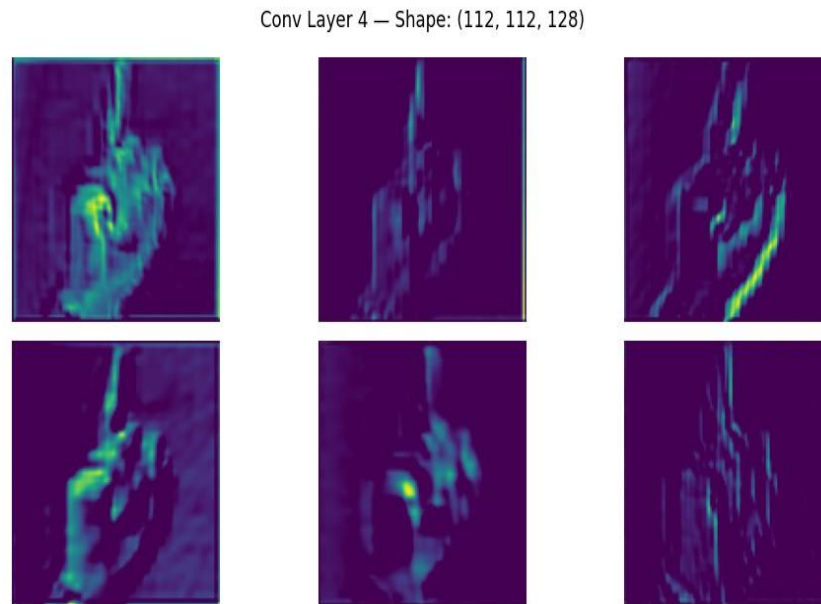


Figure 4 - VGG16 feature maps for a sample sign language image — showing first 4 convolutional layer activations.

Observation: Early convolutional layers detect low-level features — edges, contours, and intensity gradients. Intermediate layers combine these into more abstract shape representations. The hand gesture's boundary and finger separation are highlighted, while background regions are suppressed.

8.2 Pooling Layer Visualization

Max-pooling layers further compress the feature maps, retaining the strongest local activations while reducing spatial dimensions. Visualizations of the first three pooling layers show progressive abstraction with minimal information loss for salient regions.

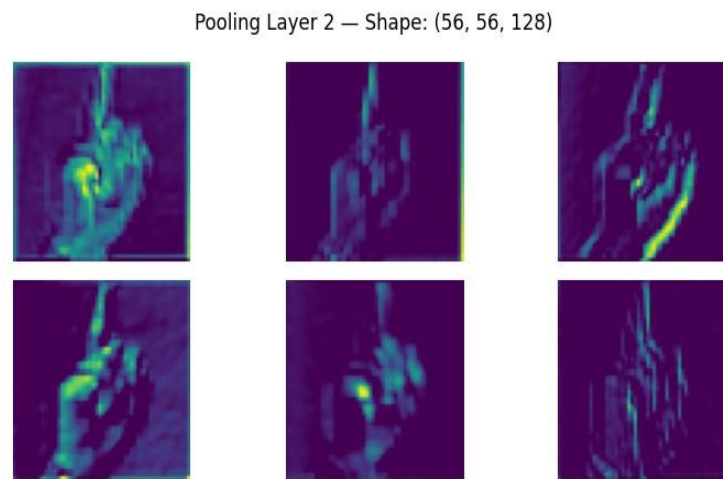


Figure 5 - Pooling layer feature maps — showing progressive spatial compression across the first three MaxPool layers.

8.3 Guided Backpropagation

A lightweight CNN (Conv2D(16) → MaxPool → Conv2D(32) → Flatten → Dense(26)) was trained for 5 epochs on the Sign Language MNIST training set, achieving 99.85% training accuracy. Guided backpropagation was then applied to compute input-gradient saliency maps for test samples.

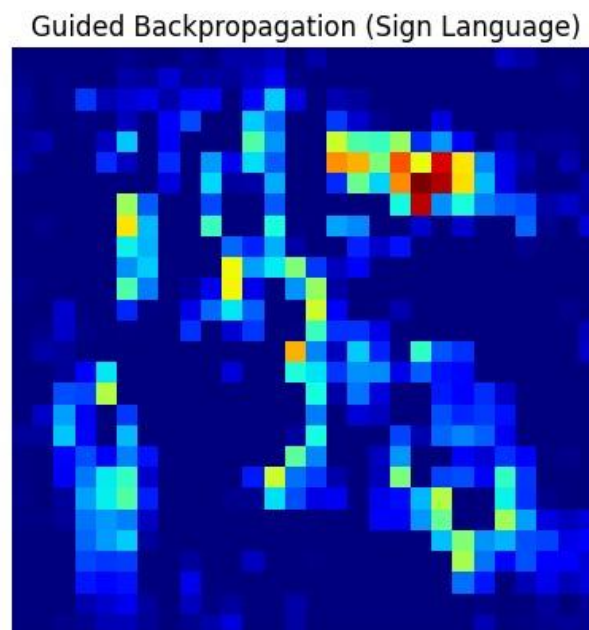


Figure 6 - Guided backpropagation saliency maps — showing which input pixels the model attends to for test samples.

Observation: Saliency maps exhibit strong activations along finger edges and knuckle contours — precisely the regions that distinguish different hand signs. Background pixels produce near-zero gradients, demonstrating that the network has learned to selectively attend to gesture-discriminative features rather than spurious background correlations.

9. Autoencoder for Unsupervised Representation Learning

9.1 Architecture

A fully-connected autoencoder was constructed with a 784-dimensional input, a bottleneck encoder layer, and a 784-dimensional sigmoid decoder layer. Three encoding dimensions were compared: 256, 128, and 64. The model was trained for 100 epochs using Adam with binary cross-entropy loss, using the training samples as both inputs and reconstruction targets.

9.2 Results and Observations

Table 6: Autoencoder Reconstruction Quality by Encoding Dimension

Encoding Dim	Reconstruction Quality	Details
256	Excellent	Sharp, well-preserved finger shapes; slight blurring only
128	Good	Noticeable blurring; shapes recognizable; fine details partially lost
64	Poor	Very blurry; gestures hard to distinguish; fingers merge

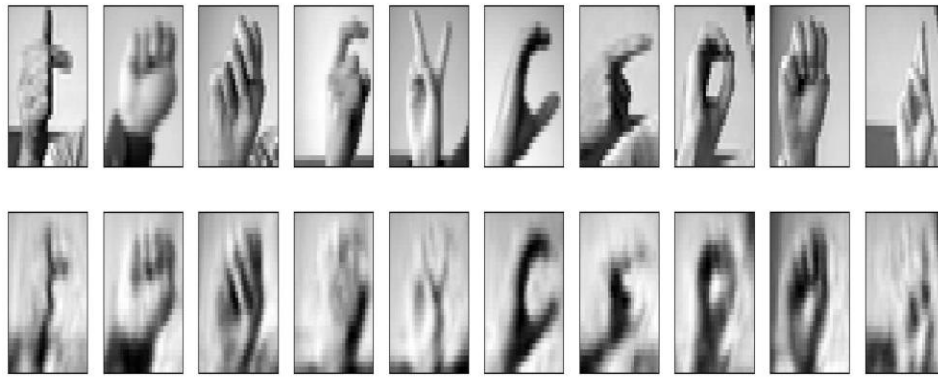


Figure 7 -Autoencoder reconstructions — top row: original images; bottom row: reconstructions at encoding dimension 128.

- **Encoding dimension 256** is recommended for near-lossless reconstruction.
- **Encoding dimension 128** provides a manageable trade-off between compression and quality.
- **Encoding dimension 64** and below result in an undercomplete representation, causing significant information loss and making gesture identification unreliable.

10. Discussion

Several cross-cutting themes emerge from the experimental results:

- **Architecture-Data Resolution Match:** The most critical factor for model performance is alignment between architecture complexity/receptive field and input resolution. LeNet and custom CNNs operating directly on 28×28 inputs achieve near-perfect accuracy, while large models (AlexNet, VGGNet, GoogLeNet) designed for 224×224 + images fail on upscaled 112×112 sign images.
- **ResNet Exception:** ResNet's residual connections enable it to converge despite the resolution mismatch, demonstrating the power of skip connections to maintain gradient flow in deep networks.
- **Learning Rate Sensitivity:** Learning rates above 0.001 cause divergence in simple MLPs.

- **Regularization Hierarchy:** Dropout(0.2) > Early Stopping > L2 > Data Augmentation > Dropout(0.3) in terms of final test accuracy in this study.
- **Autoencoder Compression Limits:** A 64-dimensional bottleneck is insufficient to preserve the spatial structure of sign language gestures. 256 dimensions represent a near-lossless lower bound for this dataset.

11. Conclusion

This assignment presented a comprehensive benchmarking study of deep learning techniques on the Sign Language MNIST dataset. Key findings include:

- MLPs can achieve near-perfect accuracy (99.98%) given appropriate learning rates and sufficient training.
- CNN architectures dramatically outperform MLPs through spatial feature extraction.
- Task-appropriate model selection is paramount — LeNet and lightweight CNNs excel while large-scale architectures fail without sufficient data and resolution.
- ResNet's residual connections provide robustness against vanishing gradients.
- Guided backpropagation confirms that learned models attend to gesture-discriminative regions (finger edges and knuckle contours).
- Autoencoders with ≥ 128 encoding dimensions provide acceptable sign language image reconstruction.

Future work should explore transfer learning with domain-appropriate pre-training, dynamic gesture recognition using sequential models (LSTM, Transformer), and multi-hand gesture datasets.

12. References

- Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *NIPS*, 2012.
- M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," *ECCV*, 2014.
- K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *ICLR*, 2015.
- C. Szegedy et al., "Going deeper with convolutions," *CVPR*, 2015.
- K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CVPR*, 2016.
- D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *ICLR*, 2015.
- N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *JMLR*, vol. 15, pp. 1929–1958, 2014.
- J. T. Springenberg, A. Dosovitskiy, T. Brox, and R. Riedmiller, "Striving for simplicity: The all convolutional net," *ICLR Workshop*, 2015.
- Kaggle Sign Language MNIST Dataset. Available:
<https://www.kaggle.com/datamunge/sign-language-mnist>